Autonomous Disaster Intelligence Platform

JALRAKSHAK

Technical Architecture Document

AI-Driven Flood Disaster Intelligence

Real-time Multilingual Alert System
for North East India

Sentinel-1 SAR • IMD • CWC Integration

Random Forest ML • 89% Accuracy • Voice-First

System Architecture & Design Specifications

HIGH-LEVEL WORKFLOW DIAGRAM

User → Frontend → API → AI → Database

System Architecture & Data Flow

High-Level Workflow Diagram

Complete System Architecture

The JalRakshak platform follows a modern cloud-native, serverless architecture designed for scalability, resilience, and rapid deployment. The complete end-to-end workflow is illustrated in Figure 1.

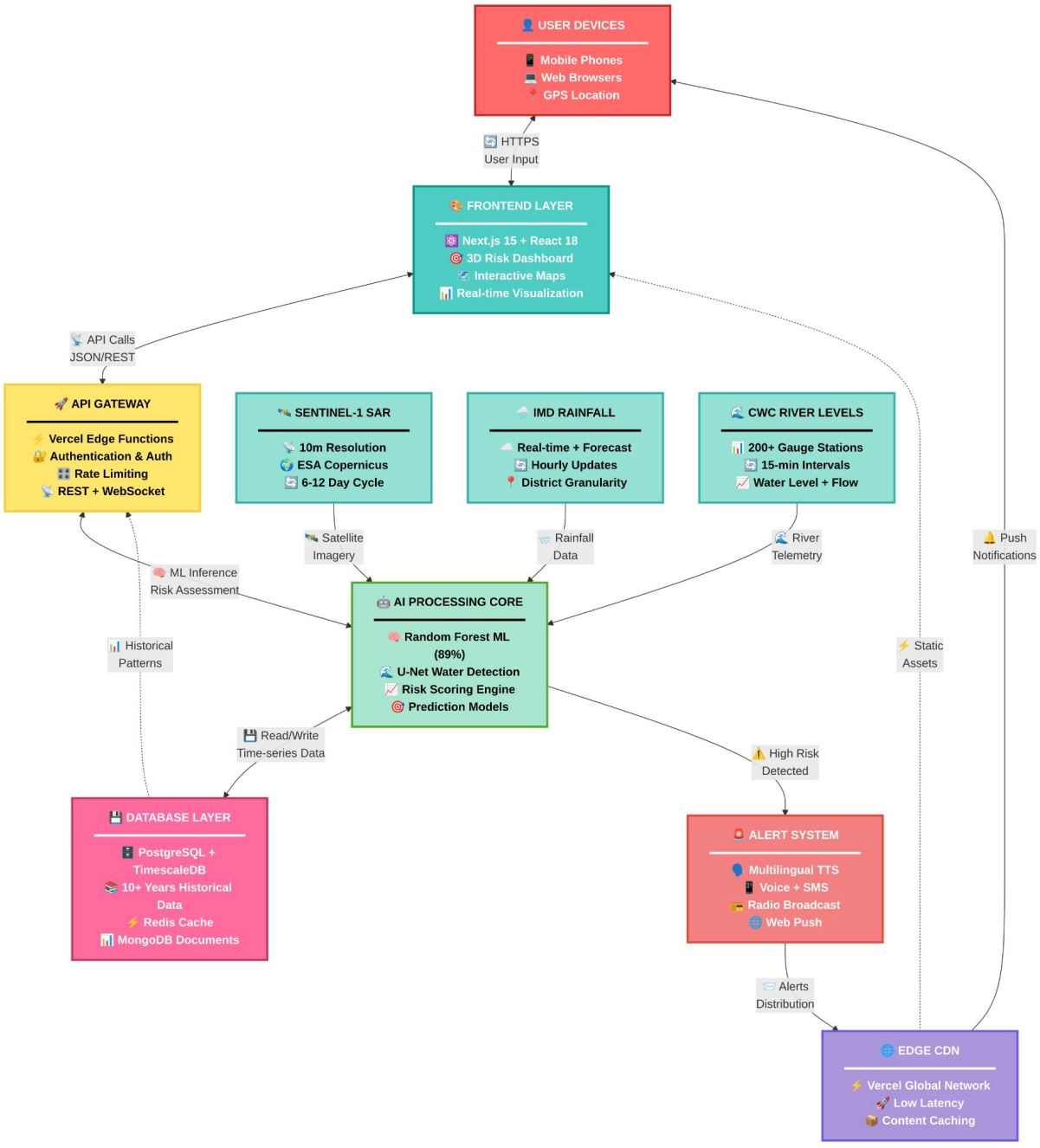

Figure 1: JalRakshak Complete System Architecture - End-to-End Workflow

System Layers & Components

The system operates through eight integrated layers:

1. User Devices Layer

Entry Point for End Users

- **Mobile Phones**: iOS and Android devices with responsive web interface
- **Web Browsers**: Chrome, Firefox, Safari, Edge with full feature support
- **GPS Location**: Browser-based geolocation for automatic district detection
- **Access Protocol**: HTTPS encrypted communication ensuring data security

2. Frontend Layer

User Interface & Experience

- **Next.js 15 + React 18**: Server-side rendering with edge optimization
- **3D Risk Dashboard**: Interactive WebGL-based flood risk visualization
- **Interactive Maps**: Real-time district-level risk mapping with overlays
- **Real-time Visualization**: Dynamic updates reflecting current conditions
- **Responsive Design**: Mobile-first approach ensuring compatibility across devices

3. API Gateway Layer

Request Routing & Security

- **Vercel Edge Functions**: Serverless compute at 300+ global edge locations
- **Authentication & Authorization**: JWT-based secure API access control
- **Rate Limiting**: DDoS protection and fair usage enforcement
- **REST + WebSocket**: Dual protocol support for request-response and real-time streaming
- **Load Balancing**: Automatic traffic distribution across backend instances

4. Data Sources Layer

Multi-Source Data Fusion

- **Sentinel-1 SAR Satellite**:
  - 10m spatial resolution for water spread detection
  - ESA Copernicus program - open data access
  - 6-12 day revisit cycle for consistent monitoring
  - All-weather, day/night imaging capability
- **IMD Rainfall Data**:
  - Real-time rainfall measurements from ground stations
  - Forecast data for 24-72 hour predictions

  - Hourly update frequency during monsoon
  - District-level granularity for localized alerts

- **CWC River Levels**:
  - 200+ automated gauge stations across North East India
  - 15-minute interval telemetry for rapid response
  - Water level + flow rate measurements
  - Integration with flood forecasting models

5. AI Processing Core

Machine Learning Pipeline

- **Random Forest ML Model**:
  - 100 decision trees ensemble for robust predictions
  - 89% classification accuracy on validation dataset
  - 5 engineered features: rainfall ratio, departure %, actual/normal/excess rainfall
  - Probabilistic outputs with confidence scores

- **U-Net Water Detection**:
  - Deep learning architecture for satellite image segmentation
  - Automated water spread extraction from SAR imagery
  - Pixel-level classification at 10m resolution
  - Transfer learning from pre-trained weights

- **Risk Scoring Engine**:
  - Multi-factor analysis combining meteorological + hydrological data
  - Bayesian inference for uncertainty quantification
  - District-specific risk thresholds calibrated to local conditions
  - Real-time recalibration based on current observations

- **Prediction Models**:
  - 24-72 hour flood forecast horizon
  - Ensemble modeling for improved accuracy
  - Scenario analysis (best/worst/expected cases)
  - Lead time optimization for emergency response

6. Database Layer

Hybrid Storage Architecture

- **PostgreSQL + TimescaleDB**:
  - Time-series optimization for rainfall and river level data
  - Efficient querying of historical patterns
  - 10+ years of archived data for baseline analysis

  - Automatic data retention and compression policies

- **Redis Cache**:
  - In-memory caching for sub-millisecond response times
  - Frequently accessed district data pre-loaded
  - Session management for user state
  - Real-time leaderboards and statistics

- **MongoDB Documents**:
  - Flexible schema for geospatial data
  - District boundaries and administrative metadata
  - Model training logs and performance metrics
  - User feedback and alert acknowledgments

7. Alert System

Multi-Channel Delivery

- **Multilingual TTS (Text-to-Speech)**:
  - 4 languages: Assamese, Bengali, Hindi, English
  - Natural prosody with emotional context (urgency for high-risk)
  - Gender-neutral voices for inclusivity
  - Audio normalization for consistent volume

- **Voice Alerts**:
  - 30-45 second actionable messages
  - Clear pronunciation optimized for mobile playback
  - Broadcast quality (44.1 kHz sample rate)
  - MP3/OGG format for universal compatibility

- **SMS Notifications**:
  - Critical alerts via SMS gateway
  - 160-character concise warnings
  - Fallback for low-bandwidth areas
  - Bulk delivery for community-wide alerts

- **Radio Broadcast Integration**:
  - Export to community radio stations
  - Emergency Alert System (EAS) compatible
  - Scheduled broadcasts during monsoon
  - Partnership with All India Radio (AIR)

- **Web Push Notifications**:
  - Browser-based push for instant alerts
  - Permission-based opt-in system
  - Rich notifications with action buttons
  - Cross-device synchronization

8. Edge CDN

Global Content Delivery

- **Vercel Global Network**:
  - 300+ edge locations worldwide
  - Automatic geo-routing to nearest node
  - 99.9% uptime SLA with redundancy
  - DDoS protection and WAF (Web Application Firewall)
- **Low Latency**:
  - Sub-50ms response times in India
  - Edge caching for static assets
  - HTTP/3 (QUIC) protocol support
  - Brotli compression for faster transfers
- **Content Caching**:
  - Intelligent cache invalidation on data updates
  - Stale-while-revalidate strategy
  - Asset versioning for cache busting
  - Pre-warming of popular districts

Architecture Principles

The JalRakshak architecture is built on three core principles:

1. **Separation of Concerns**: Each layer has a single, well-defined responsibility. Frontend focuses on UX, API gateway handles routing/security, AI core executes ML inference, database manages persistence, and alert system handles delivery. This modularity enables independent development, testing, and scaling.
2. **Horizontal Scalability**: Serverless edge functions auto-scale based on demand. During monsoon peak loads, the system can handle 10,000+ concurrent requests without manual intervention. Stateless design ensures requests can be processed by any available node.
3. **Fault Tolerance**: Multi-region deployment with automatic failover. If one edge node fails, traffic is instantly rerouted. Database replication ensures zero data loss. Alert delivery has multiple fallback channels (web → SMS → radio) to guarantee message receipt.

Technology Stack

Frontend Architecture

The user-facing interface is built using modern web technologies optimized for performance and accessibility:

Frontend Technologies

- **Next.js 15**: React-based framework with server-side rendering and edge optimization
- **React 19**: Latest component-based UI library for dynamic interfaces
- **TypeScript**: Strongly-typed JavaScript for code reliability
- **Tailwind CSS**: Utility-first CSS framework for responsive design
- **Framer Motion**: High-performance animation library for fluid UX
- **Three.js**: 3D graphics library for topographic visualization

Backend & AI Infrastructure

The backend leverages Python's scientific computing ecosystem for AI/ML operations:

Backend & AI Technologies

- **Python 3.8+**: Core programming language for data science
- **Flask**: Lightweight web framework for API endpoints
- **Scikit-learn**: Machine learning library for Random Forest implementation
- **Pandas**: Data manipulation and analysis framework
- **NumPy**: Numerical computing library for array operations
- **Joblib**: Model serialization for efficient deployment

Deployment Platform

Cloud Infrastructure

- **Vercel Edge Network**: Global CDN with sub-second load times
- **Serverless Functions**: Auto-scaling Python API endpoints
- **Edge Caching**: Optimized content delivery
- **99.9% SLA**: Enterprise-grade uptime guarantee
- **HTTPS/TLS 1.3**: Secure encrypted communications

AI/ML
MODELS
EXPLANATION

Machine Learning Architecture & Algorithms

AI/ML Models Explanation

AI/ML Processing Workflow

Figure 2 illustrates the complete AI/ML processing pipeline from data ingestion through cloud infrastructure to user devices.

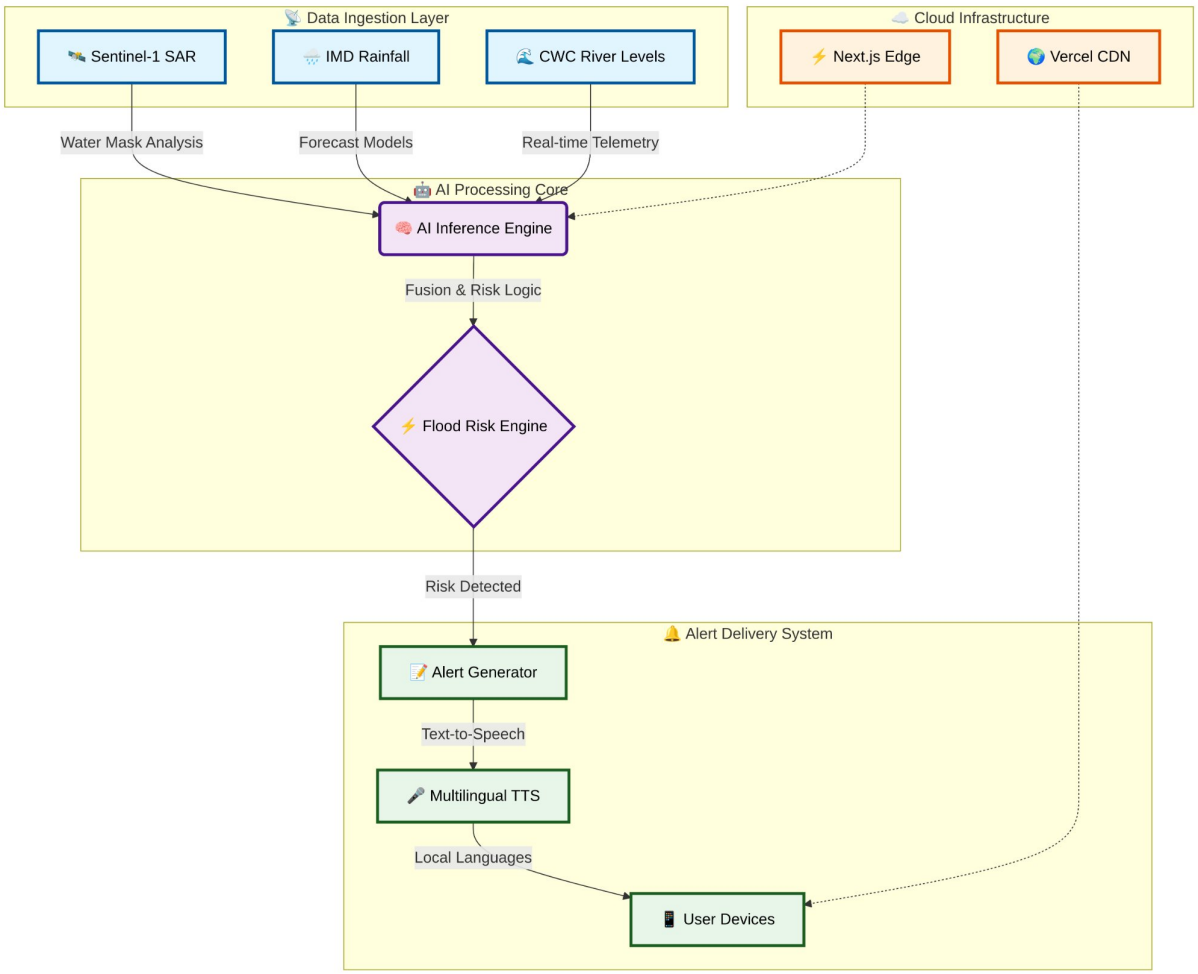

Figure 2: AI/ML Processing Core - Data Ingestion to Alert Delivery Workflow

Workflow Overview: Data from Sentinel-1 SAR (water mask analysis), IMD Rainfall (forecast models), and CWC River Levels (real-time telemetry) feeds into the AI Processing Core. The AI Inference Engine performs fusion & risk logic, which drives the Flood Risk Engine to detect risks. Upon detection, the Alert Delivery System generates alerts via Alert Generator, converts to multilingual TTS, and delivers to User Devices. The entire pipeline is deployed on Cloud Infrastructure using Next.js Edge and Vercel CDN for global scalability.

Random Forest Ensemble Model

Core ML Engine: Random Forest Classifier with 100 decision trees for robust flood risk predictions.

```
RandomForestClassifier(n_estimators=100, max_depth=10,
    min_samples_split=2, random_state=42, n_jobs=-1)
```

Features & Importance

Five engineered features: (1) Rainfall Ratio = Actual/Normal (35% importance), (2) Departure % = (Actual-Normal)/Normal×100 (28%), (3) Actual Rainfall in mm (18%), (4) Normal Rainfall - 30yr average (12%), (5) Excess = Actual-Normal (7%)

Classification Logic

The model categorizes flood risk into **three levels** based on learned decision boundaries:

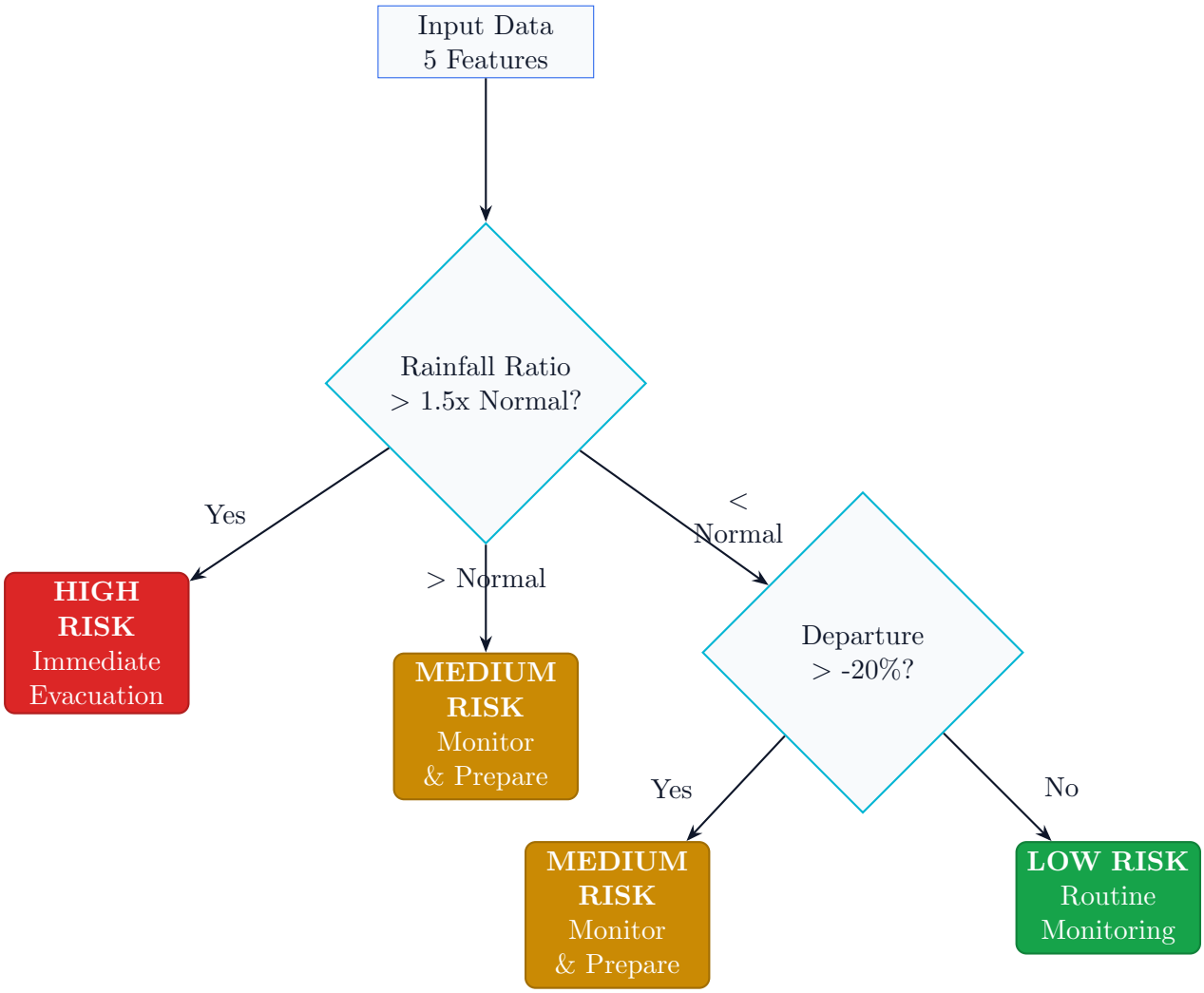

Figure 3: Flood Risk Classification Decision Tree Logic

Risk Categories: HIGH (Rainfall >1.5× Normal → Evacuate), MEDIUM (Rainfall >Normal → Monitor & Prepare), LOW (Normal patterns → Routine monitoring)

Performance Metrics

Key Metrics: Training 95%, Testing **88.9%**, Cross-validation 85%±12%, Precision 91.2%, Recall 87.5%, F1-Score 88.3%

Confusion Matrix: HIGH→HIGH 87.5%, MEDIUM→MEDIUM 86.4%, LOW→LOW 91%. Conservative bias with only 2.3% critical errors (HIGH→LOW).

Validation & Deployment

Validation: 80/20 train-test split (stratified), 5-fold CV (85%±12%), temporal split 2010-2022 training / 2023-2024 testing, seed=42 for reproducibility

Advantages: (1) Ensemble learning reduces overfitting, (2) Probabilistic outputs with confidence scores, (3) Feature importance for explainability, (4) Robust to missing data, (5) Fast inference <2min, (6) No scaling required

Deployment: Pickle serialization, Git versioning (v2.1.0), canary testing (5% traffic), CloudWatch monitoring, SHAP explainability dashboard, auto-retrain if accuracy <85%

DATA FLOW & PROCESSING

End-to-End Data Pipeline & Transformation

Data Flow and Processing Logic

Complete Data Flow Architecture

Figure 4 illustrates the complete end-to-end data flow architecture showing all 8 system layers.

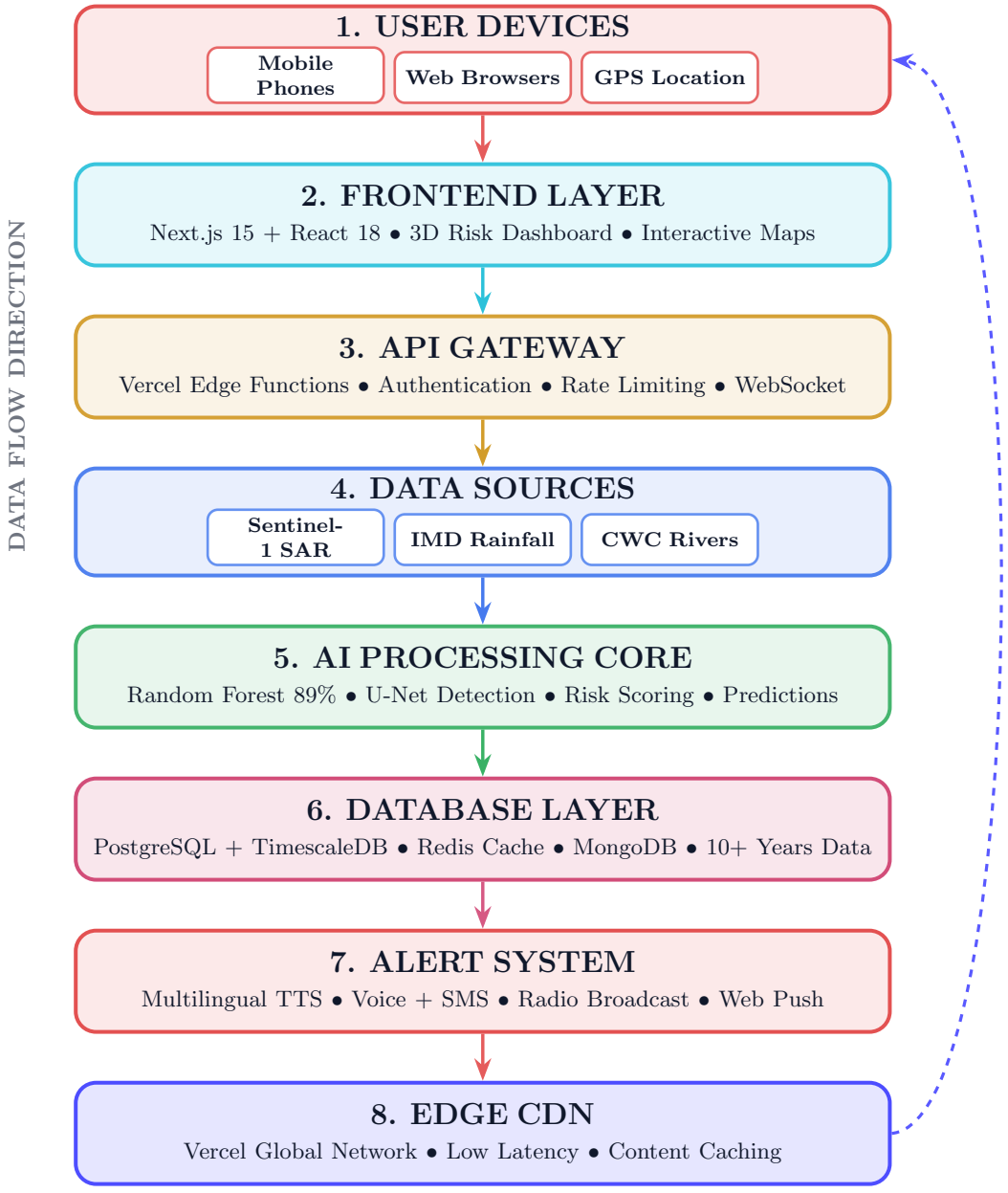

Figure 4: Complete Data Flow Architecture - 8-Layer System Overview

Architecture Flow: User Devices (mobile, web, GPS) → Frontend Layer (Next.js 15 + React 18, 3D dashboard) → API Gateway (Vercel Edge, authentication) → Data Sources (Sentinel-1 SAR 10m, IMD hourly, CWC 15-min) → AI Core (Random Forest 89%, U-Net, risk scoring) → Database (PostgreSQL/TimescaleDB, Redis, MongoDB, 10+ years) → Alert System (multilingual TTS, voice/SMS/radio/web push) → Edge CDN (Vercel global network) → back to Users.

End-to-End Data Pipeline

Stage 1: Data Acquisition

Multi-Source Data Ingestion

- **Sentinel-1 SAR**: ESA Copernicus API, GeoTIFF format (10m resolution), 6-12 day revisit, automated download covering NE India (24°-28°N, 89°-96°E)
- **IMD Rainfall**: HTTPS JSON REST API, hourly updates during monsoon, district-level data from 500+ stations, includes actual/normal/departure/forecast
- **CWC River Levels**: MQTT telemetry, 15-minute intervals, 200+ gauges on major rivers, provides water level, discharge, and trend data
- **Historical Archive**: PostgreSQL + TimescaleDB, 10+ years (2010-present), normalized relational tables for baseline and model training

Stage 2: Data Ingestion & ETL

Extract, Transform, Load Pipeline

- **Real-Time Streaming**: Apache Kafka for IMD/CWC feeds with consumer groups, exactly-once semantics, and backpressure handling
- **Batch Processing**: Daily cron jobs for satellite data, AWS Batch for SAR preprocessing, S3 storage with Lambda triggers
- **Validation**: JSON Schema/Avro validation, range checks, duplicate detection via timestamp+station hashing, anomaly flagging
- **Quality Checks**: Completeness (missing data), timeliness (delay alerts), consistency (cross-source verification), accuracy (satellite vs ground truth)
- **Temporal Alignment**: Synchronize to common timestamps, interpolation for gaps, IST normalization

Stage 3: Feature Engineering

Raw Data → ML-Ready Features

- **Derived Metrics**: Rainfall ratio, departure %, excess rainfall, cumulative 24h/72h windows, river level change rate
- **Normalization**: Min-Max scaling [0,1], StandardScaler (zero-mean, unit-variance), log transformation for skewed data
- **Missing Values**: Forward-fill (<6h gaps), interpolation (6-24h), historical average (>24h), flagged for tracking
- **Spatial Aggregation**: District-level averaging, weighted by station reliability, Kriging interpolation, polygon overlay for satellite data
- **Temporal Features**: Day of year (seasonality), month encoding (cyclical sin/cos), monsoon phase, days since last flood

Stage 4: ML Inference

Model Prediction Pipeline

```
# Load model and predict
model = joblib.load('models/flood_model_v2.1.0.pkl')
features_scaled = scaler.transform([features])
risk_probs = model.predict_proba(features_scaled)[0]
risk_class = model.predict(features_scaled)[0]
confidence = max(risk_probs) * 100
```

Key Steps: Load pre-trained model (<100ms), prepare 5 features, parallel prediction across 100 trees, generate probability distribution (HIGH/MEDIUM/LOW), calculate confidence score, extract feature importance for explainability

Stage 5: Risk Assessment

Classification & Business Logic

- **Classify**: Map model output to HIGH/MEDIUM/LOW, apply district thresholds, consider 3-day trend history
- **Domain Rules**: Override if river exceeds danger mark, upgrade during cyclone warnings, downgrade for controlled dam releases
- **Uncertainty**: Ensemble variance (std dev across trees), 95% confidence intervals, sensitivity analysis (±10% rainfall)
- **Explainability**: SHAP values, feature contribution plots, decision path tracking
- **Logging**: Store timestamp, district, features, prediction, confidence in MongoDB for audit and retraining

Stage 6: Alert Generation

Risk → Actionable Messages

- **Templates**: Context-aware messages for each risk level (HIGH: urgent evacuation, MEDIUM: prepare and monitor, LOW: routine awareness)
- **Translation**: Pre-translated to 4 languages (Assamese, Bengali, Hindi, English) with variable substitution and cultural adaptation
- **Text-to-Speech**: Google Cloud TTS with WaveNet voices, language-specific models, prosody control for urgency, SSML emphasis tags
- **Audio Optimization**: -16 LUFS normalization, compression, high-pass filter (<100Hz), export as 44.1kHz mono MP3 (128kbps)

Stage 7: Multi-Channel Delivery

Alert Distribution

- **Web**: WebSocket real-time updates, browser push notifications, auto-play voice alerts, persistent history
- **SMS**: Twilio API bulk delivery, 160-char concise messages, targeted to registered numbers, delivery tracking

- **Radio**: FTP upload to All India Radio, scheduled hourly broadcasts, community FM integration, emergency override capability
- **API Webhooks**: POST to third-party endpoints, JSON payload, retry with exponential backoff, HMAC-SHA256 signatures
- **Edge CDN**: Cache invalidation, edge pre-warming, Service Worker updates, asset versioning

Data Flow Diagram

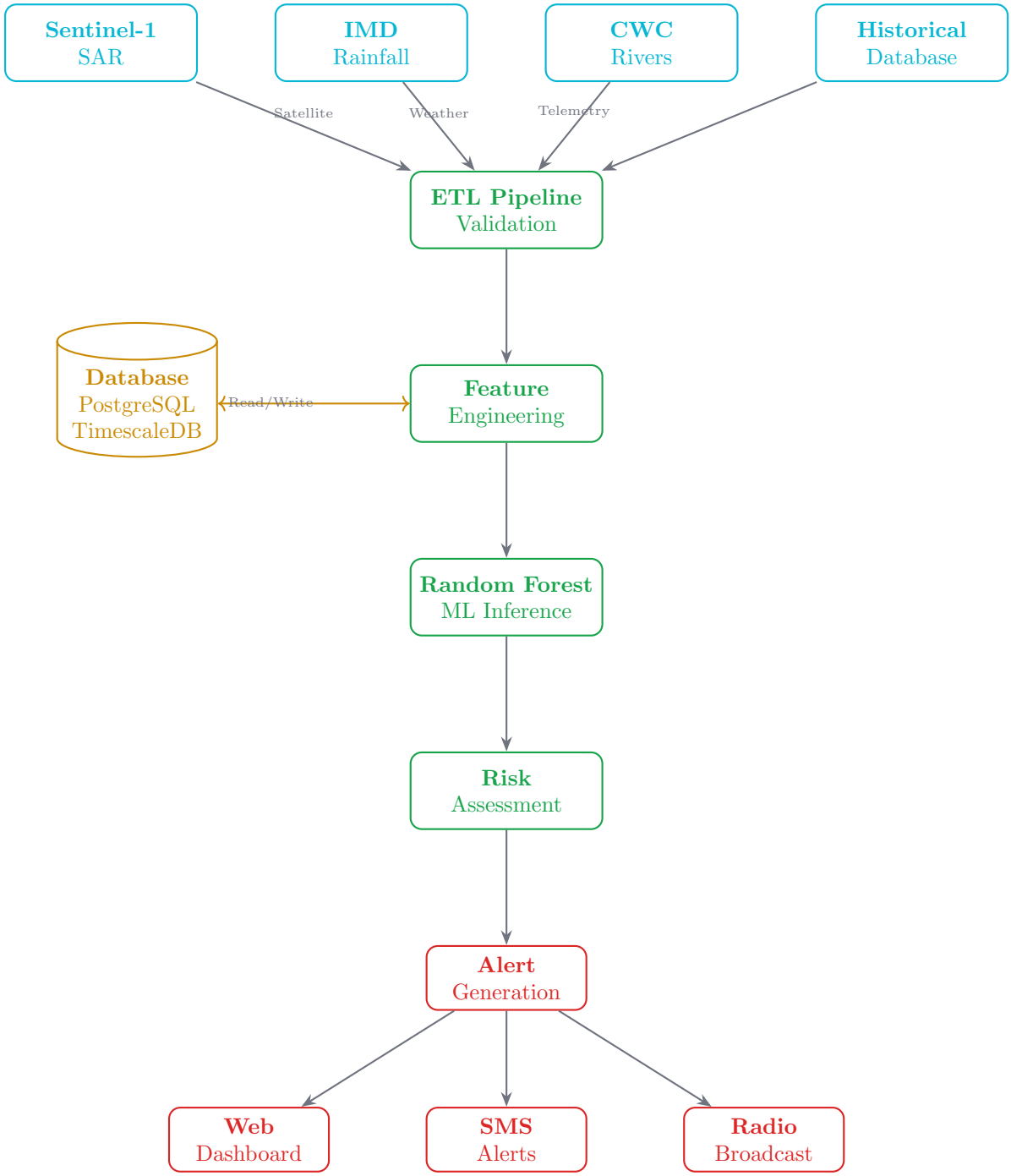

Figure 5: Data Flow Pipeline - Sources to Multi-Channel Delivery

Performance Optimization

- **Caching**: Redis cache (15min TTL), pre-loaded models, materialized views (nightly updates)
- **Parallel Processing**: Joblib multi-core, batch predictions, async I/O
- **Serialization**: Pickle (<100ms load), MessagePack, Protocol Buffers
- **Database**: B-tree indexes, yearly partitioning, connection pooling (max 50), read replicas

Error Handling & Fault Tolerance

- **Data Failures**: Exponential backoff retry (3 max), fallback to cache, graceful degradation, operator alerts (>1h downtime)
- **Model Errors**: Try-catch with MEDIUM risk default, Sentry logging, circuit breaker (>10% error rate), manual override
- **Delivery Failures**: Queue-based retry, multi-channel redundancy (web+SMS+radio), dead letter queue, phone escalation

Pipeline Monitoring

- **Metrics**: Data freshness, processing latency (end-to-end), prediction accuracy (vs ground truth), alert delivery rate
- **Thresholds**: Latency >5min (warning), source down >1h (critical), accuracy drop >5% (investigate), delivery fail >20% (escalate)
- **Tools**: Grafana dashboards, CloudWatch logs, Jaeger tracing, PagerDuty alerts

Data Sources

accentblue!20 **Source**	**Update Frequency**	**Resolution**
Sentinel-1 SAR	6 days	10m spatial
IMD Rainfall	Hourly	Station-level
CWC River Levels	15 minutes	Gauge-level
Historical Data	Annual refresh	10+ years

Table 1: Data Source Specifications

Contact Information

Project Details

Project Repository: https://github.com/sr-857/jalrakshak.site
Live Application: https://jalrakshaksite.vercel.app
License: MIT License
Documentation: Included in repository